Lab Azure Container Apps – Deploying the Database Container To Begin with the Lab

Summary

In this lab, we deploy a **MySQL database container** as a separate **Azure Container App** within the same **Container Apps Environment** as the web application. The database container is configured with environment variables, exposed only internally using **TCP port 3306**, and accessed securely by the web application.

Understanding

Azure Container Apps allow multiple containers to run inside the same **Container Apps Environment**. In this lab, one Container App hosts the **ASP.NET web application**, while another hosts the **MySQL database**. Since both apps are in the same environment, the web application can communicate with the database without exposing it to the public internet.
Container Apps also support **automatic scaling**. By default, if there is no incoming traffic, a Container App can scale down to **zero replicas**, reducing costs. If continuous availability is required, the minimum replica count can be set to **1**.
The MySQL container requires the **MYSQL_ROOT_PASSWORD** environment variable during deployment. Unlike the web application, the database uses **TCP traffic** on **port 3306** instead of HTTP.

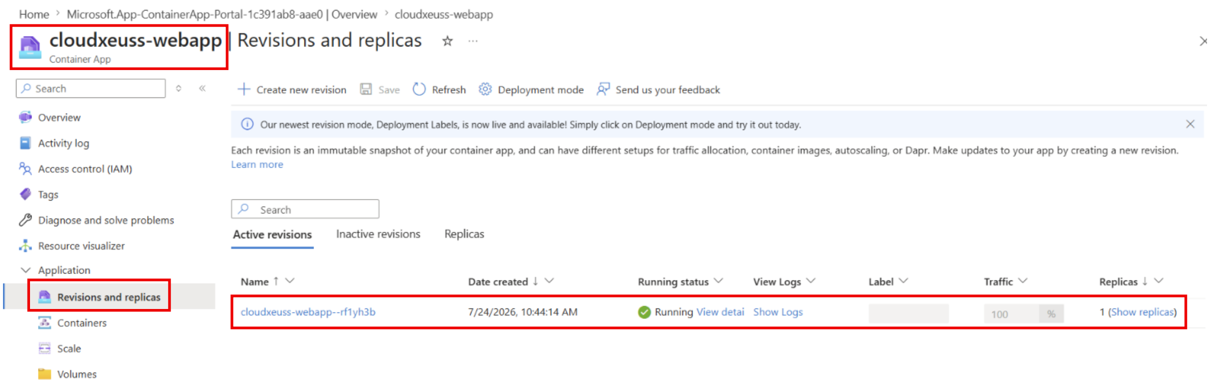
Example:

- **Container App 1:** ASP.NET Web Application (HTTP/HTTPS)
- **Container App 2:** MySQL Database (TCP 3306)
- Both run in the same **Container Apps Environment** and communicate internally.

Lab Steps

Step 1: Open the Existing Container App

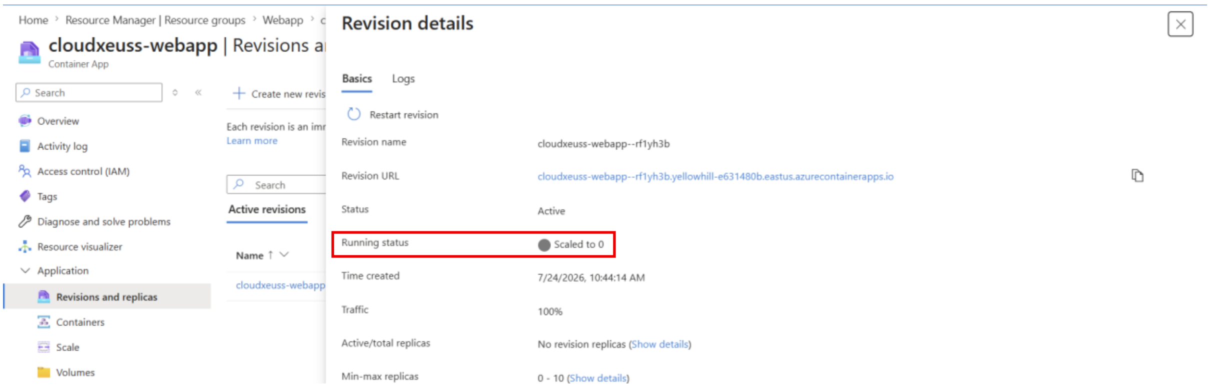
- Open the Azure Portal.
- Navigate to the deployed **Container App**.
- Open **Revisions and Replicas**.
- Verify the application revision.



Step 2: Check Replica Status

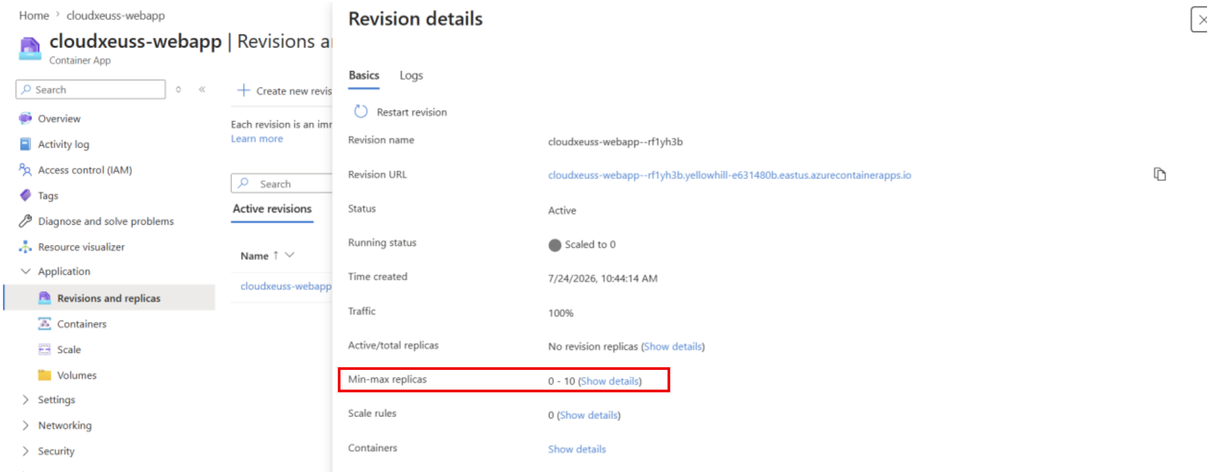
- Observe the running replicas.
- If there is no traffic, the Container App may show **0 running instances**.

- This is normal because Container Apps automatically scale down to save cost.

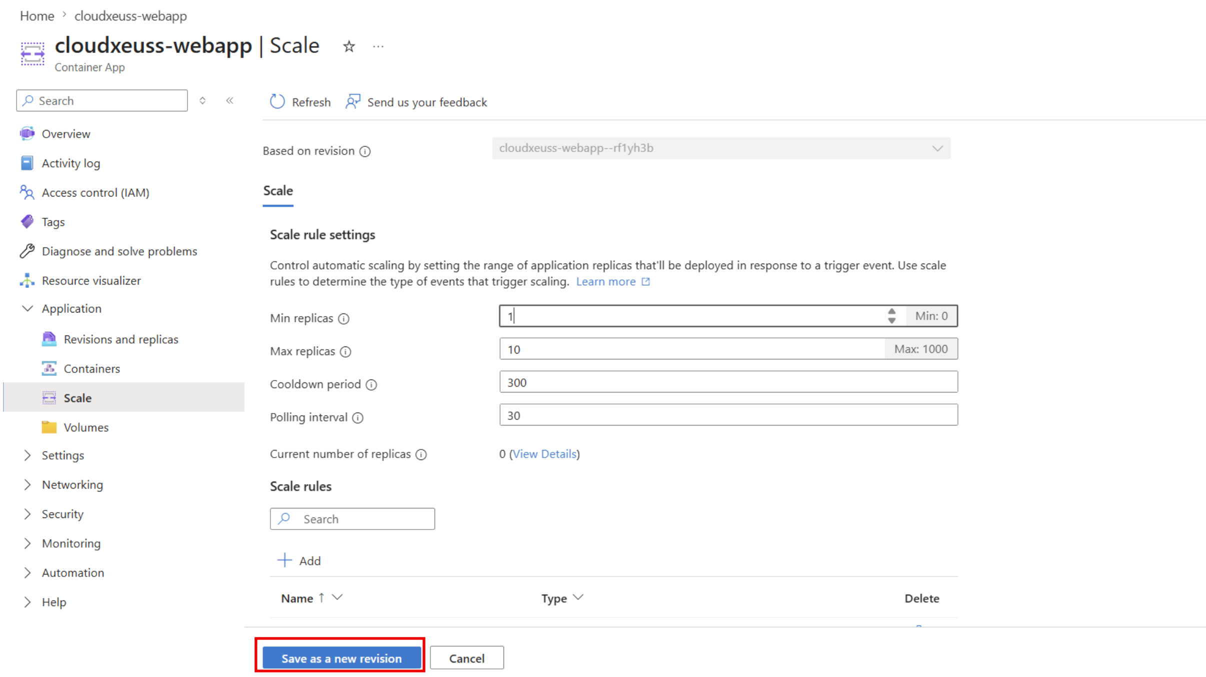


Step 3: Configure Minimum Replicas

- Click **Show Details** under Min/Max Replicas.

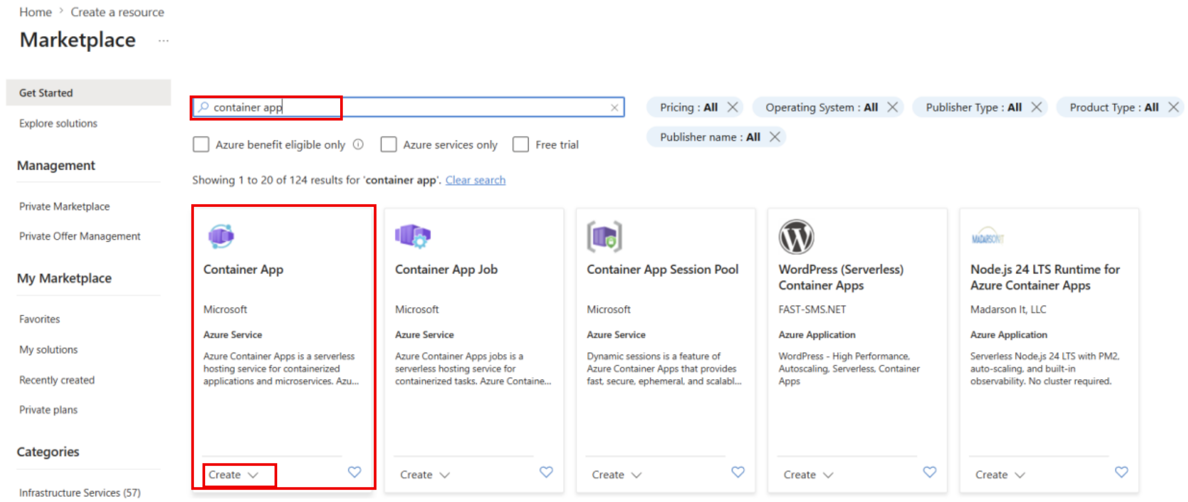


- Change **Minimum Replicas** from **0** to **1**.
- Save the configuration.
- Azure creates a **new revision**.
- Wait until the revision becomes active.

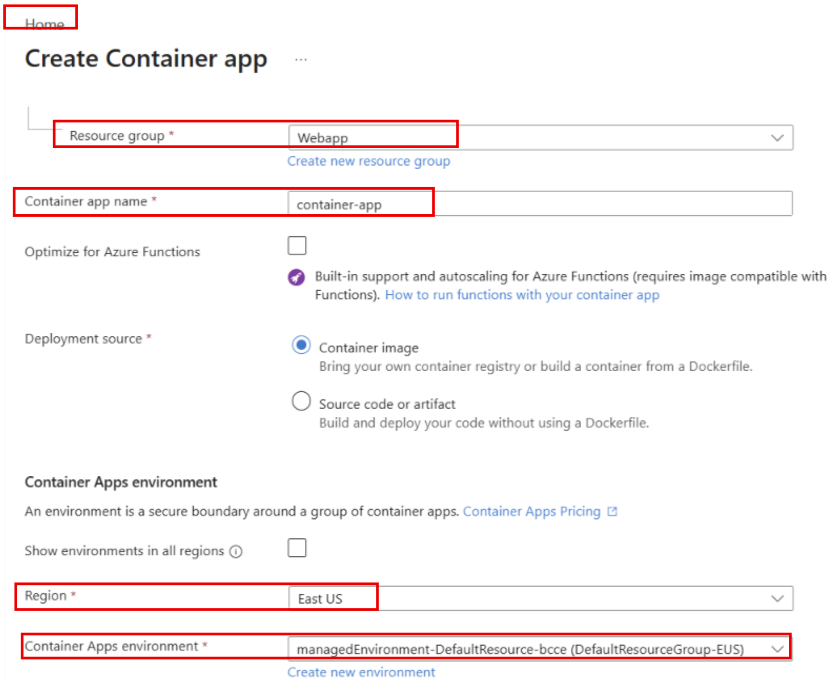


Step 4: Create a New Container App for MySQL

- Go to **Azure Portal**.
- Search for **Container Apps**.
- Click **Create**.

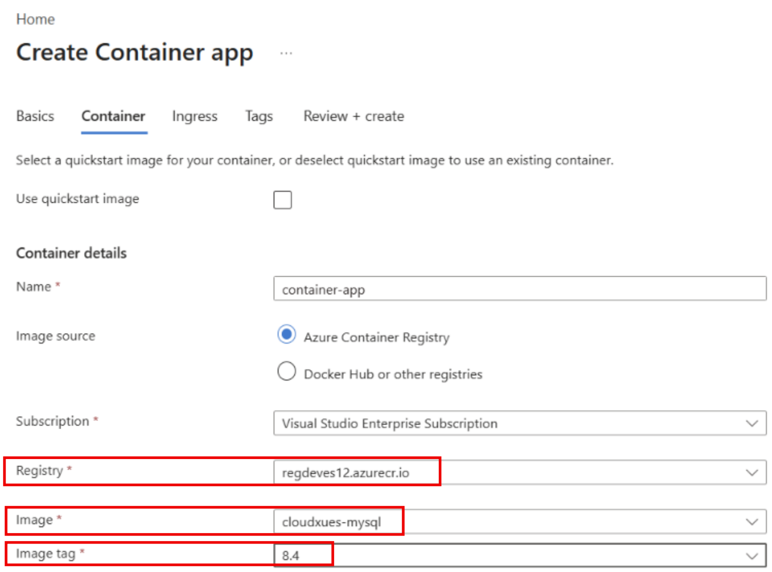


- Select the existing **Resource Group**.
- Enter the **Container App Name**
- Select **East US Region**.
- Use the same **Container Apps Environment** created earlier.



Step 5: Select the MySQL Container Image

- Click **Next: Container**.
- Choose the **Azure Container Registry (ACR)**.
- Select the **MySQL Docker Image**.
- Select the required **Image Tag**.

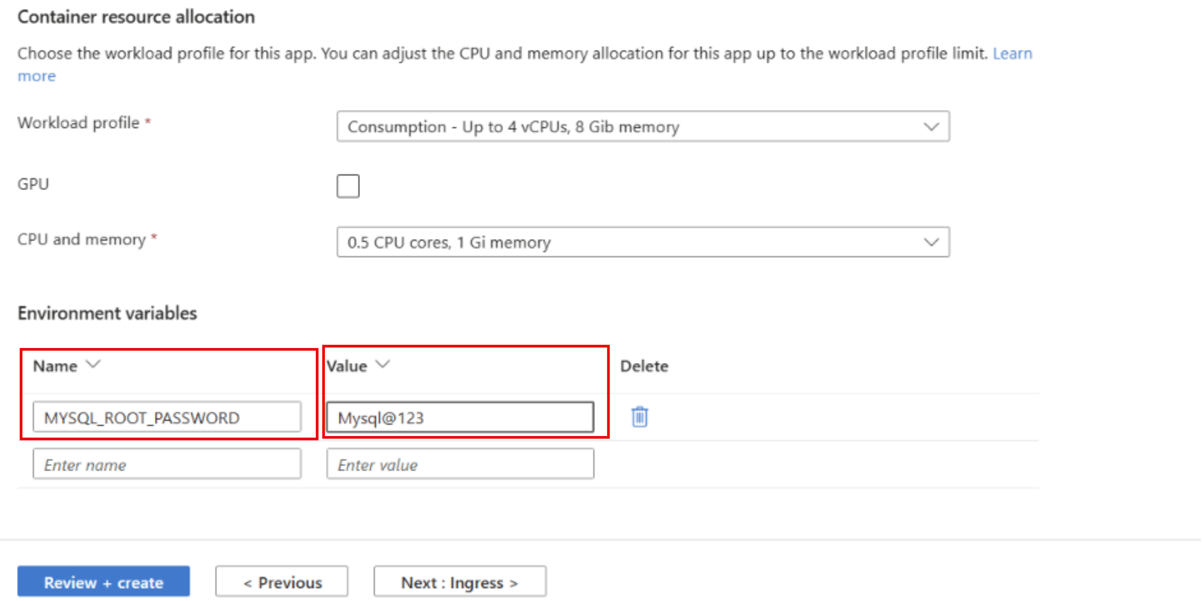


Step 6: Configure Environment Variables

- Add the required environment variable.

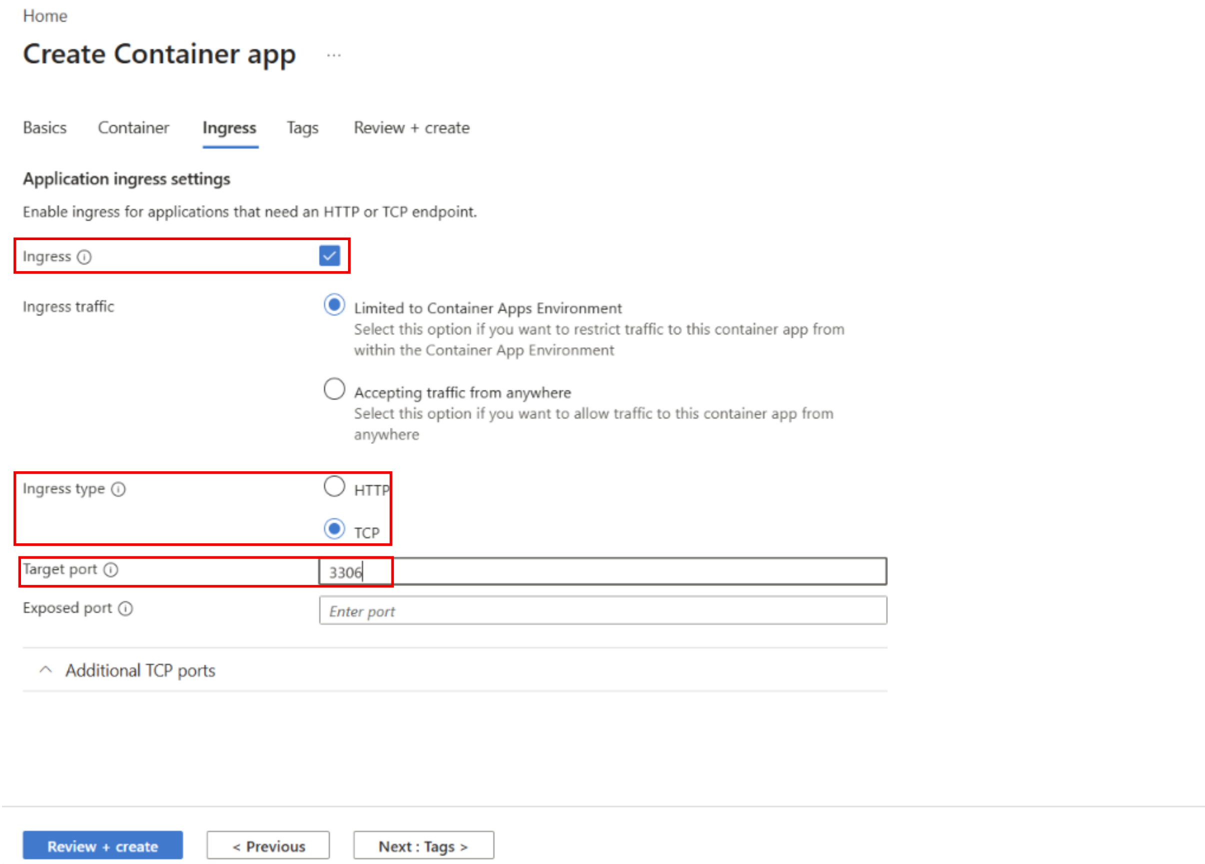
Variable	Value
MYSQL_ROOT_PASSWORD	Mysql@123

- Save the environment variable.



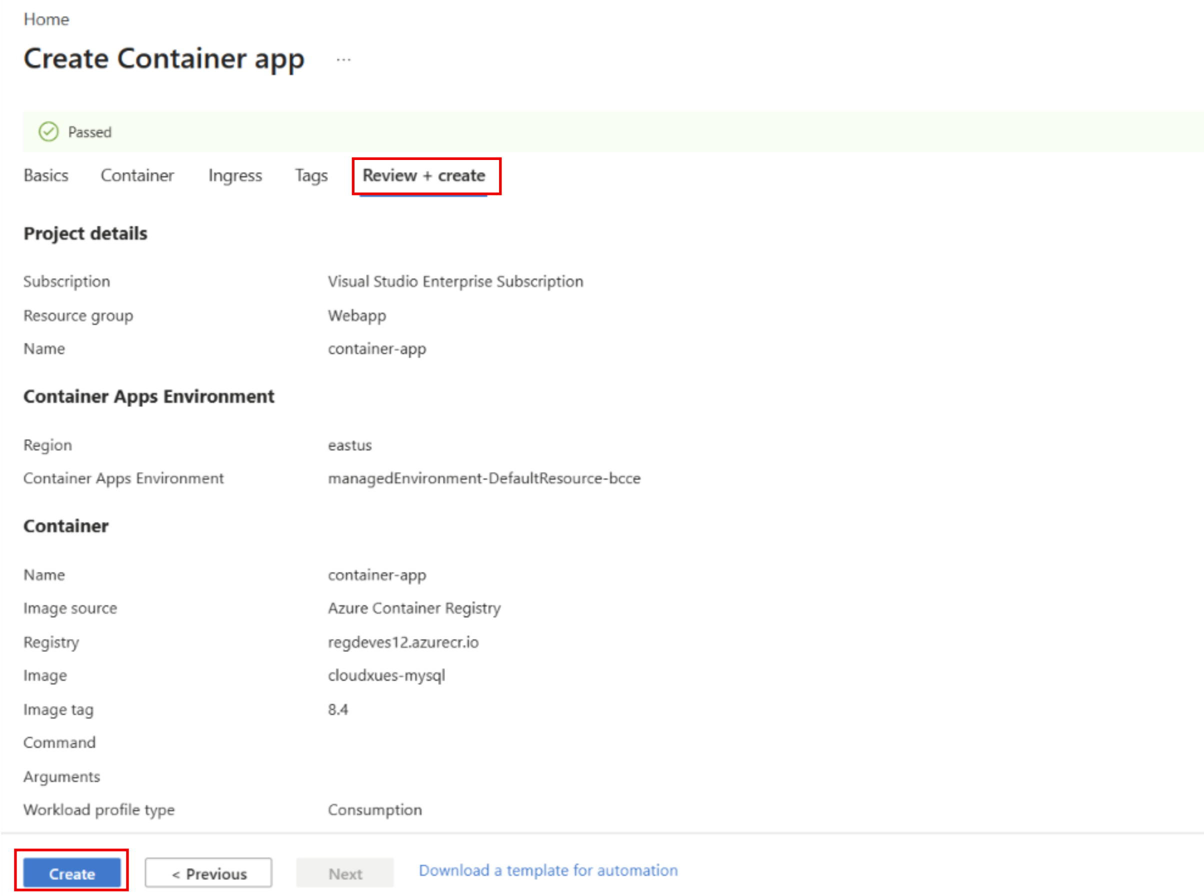
Step 7: Configure Ingress

- Enable **Ingress**.
- Choose **Limited to Container Apps Environment (Internal)**.
- Select **TCP** as the transport protocol.
- Set **Target Port** to **3306**.
- Do **not** use HTTP because MySQL communicates using TCP.

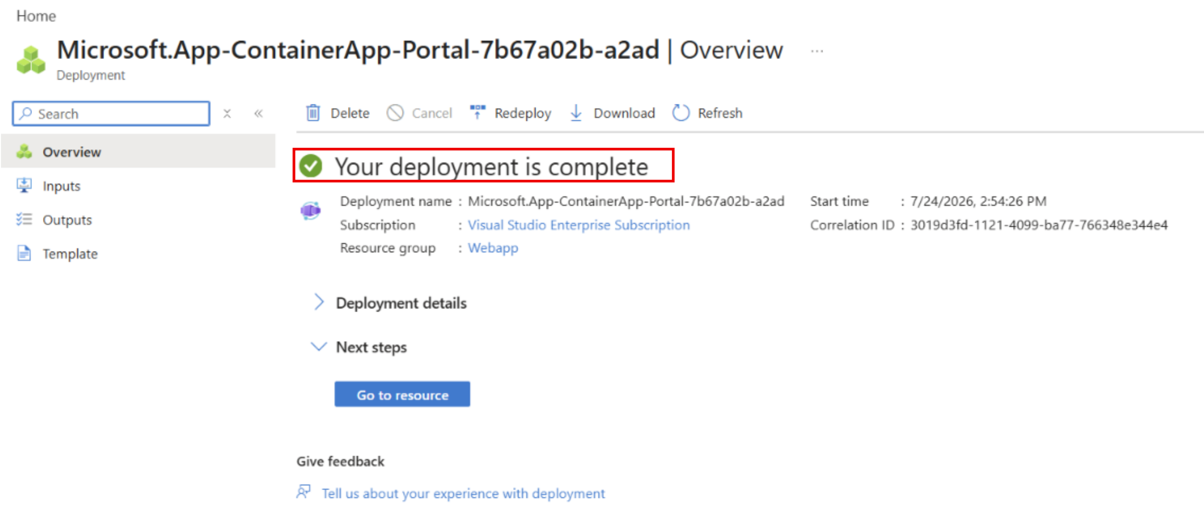


Step 8: Deploy the Database Container

- Click **Review + Create**.
- Click **Create**.

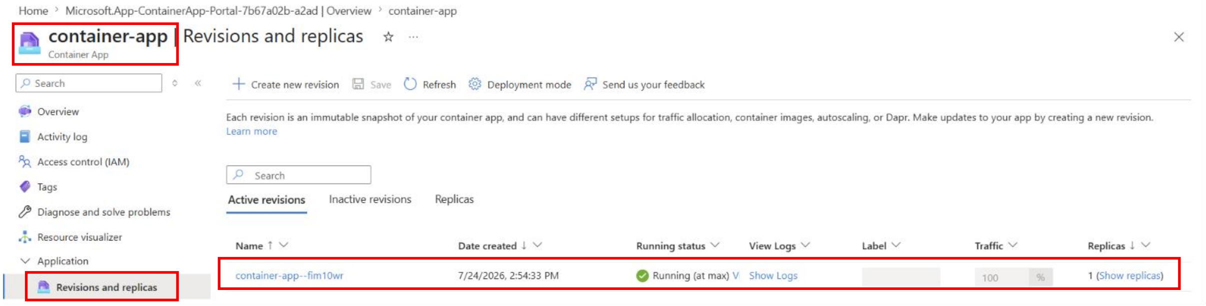


- Wait until deployment completes successfully.



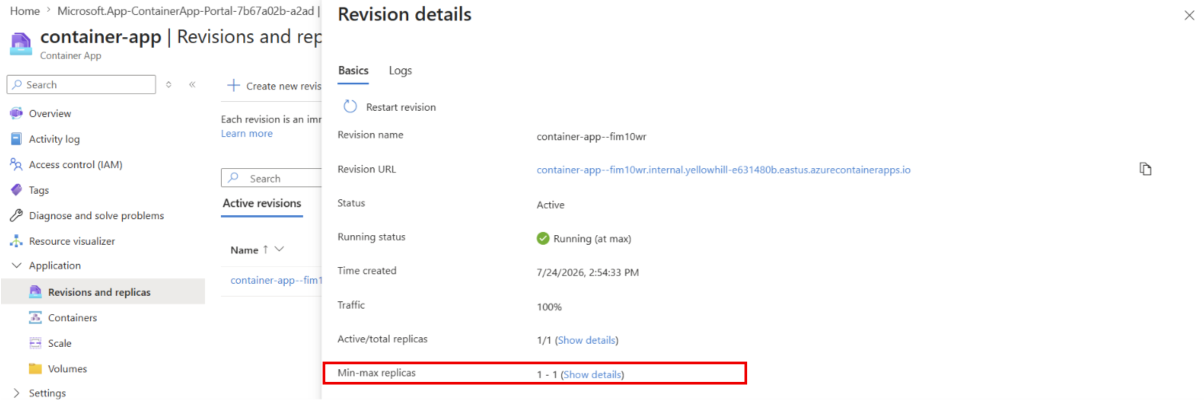
Step 9: Verify Deployment

- Open the newly created **Container App**.
- Navigate to **Revisions and Replicas**.
- Verify that the revision is active.
- Refresh the page if required.



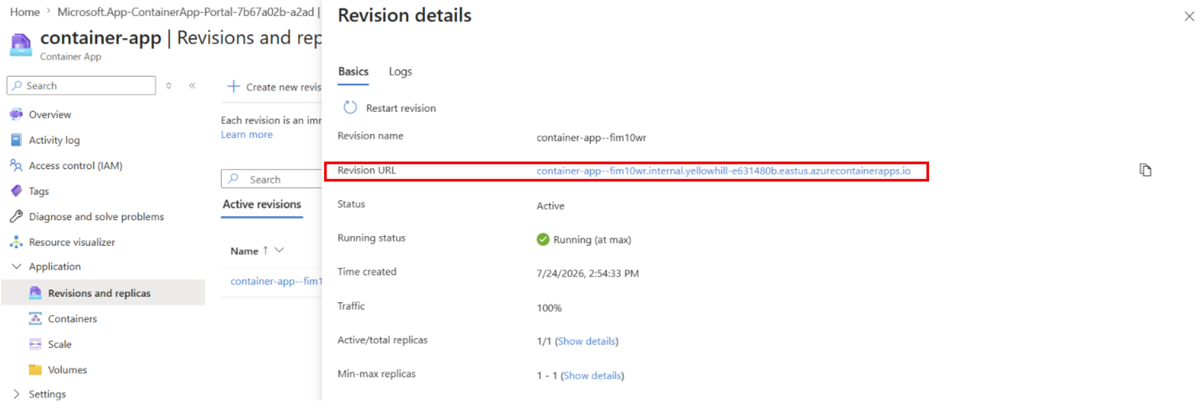
Step 10: Verify Replica Status

- Ensure the MySQL Container App is running.
- If necessary, configure **Minimum Replicas = 1** to keep one instance always active.



Step 11: Copy the Revision URL

- Open the active revision.
- Copy the generated **Revision URL**.
- This confirms the Container App has been deployed successfully.



Step 12: Test the Application

- Open the **Orders** page of the web application.
- Verify that the application retrieves data successfully.
- Confirm the web application can communicate with the MySQL database.

container-app--fim10wr.internal.yellowhill-e631480b.eastus.azurecontainerapps.io

dockerapp Home Privacy

Course Orders

OrderId	Customer	Email	Course	Amount	OrderDateUtc
5	Customer 005	customer005@cloudxeus.com	AI-9004: Azure AI Fundamentals	19.00	2026-07-22 10:34:41Z
4	Customer 004	customer004@cloudxeus.com	AZ-3056: Azure Architect	69.00	2026-07-22 10:34:41Z
3	Customer 003	customer003@cloudxeus.com	DP-6004: Fabric Analytics Engineer	59.00	2026-07-22 10:34:41Z
2	Customer 002	customer002@cloudxeus.com	AZ-1046: Azure Administrator	39.00	2026-07-22 10:34:41Z
1	Customer 001	customer001@cloudxeus.com	AZ-2045: Azure Developer Associate	49.00	2026-07-22 10:34:41Z